

CARE: Experience-Guided Atomic Corrective Execution for Vision-Language-Action Policies

Junlan Xiao^{1*}, Junwei Jiang^{1*}, Zaibin Zhang^{1*}, Yifan Wang¹, Zhongbo Zhang¹, Huchuan Lu¹, Lijun Wang^{1†}

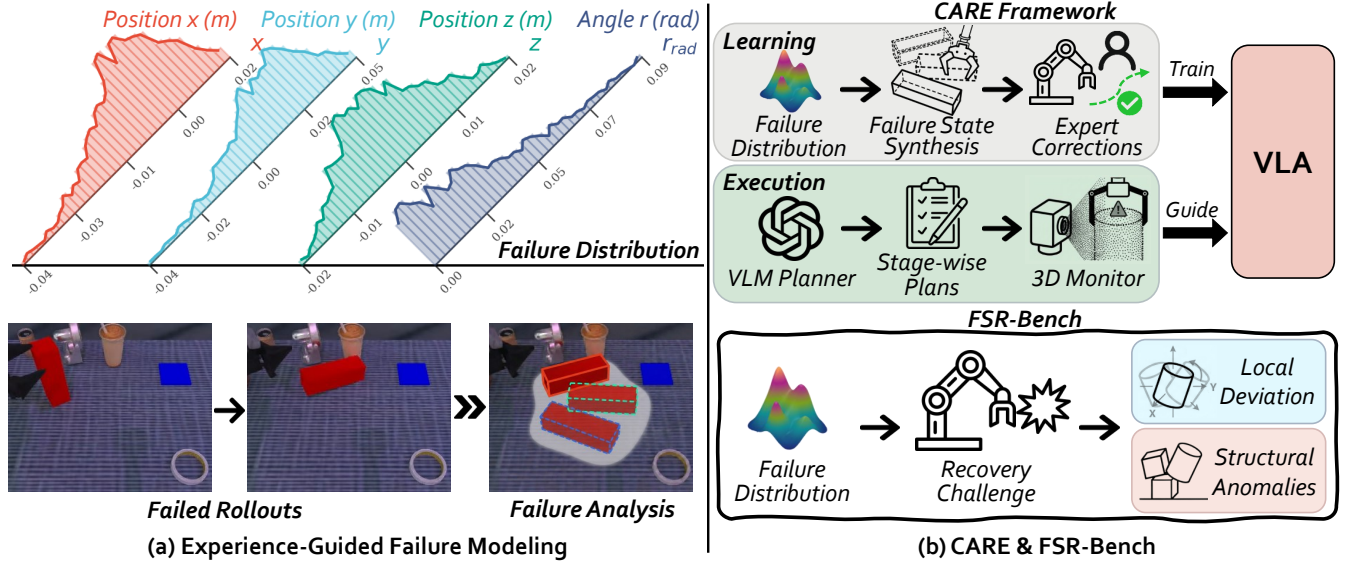


Fig. 1. Overview of CARE and FSR-Bench. (a) CARE models stage-specific failure distributions from execution experience to generate failure states and collect corrective demonstrations. (b) CARE uses these distributions for recovery learning, whereas FSR-Bench independently estimates its evaluation distribution from a mixture of nominal policies to generate shared recovery initializations.

Abstract—Vision-Language-Action (VLA) policies achieve strong performance in robotic manipulation but remain brittle once execution deviates from nominal trajectories. We propose CARE (Corrective Atomic Robotic Execution), a framework that improves recovery by learning from failures encountered during execution. Instead of generating corrective data from manually designed or random perturbations, CARE collects failed rollouts, models stage-conditioned post-failure deviations, and uses the resulting empirical distributions to synthesize representative failure states and corrective demonstrations. At inference time, CARE combines stage-wise planning with physically grounded 3D monitoring to trigger atomic adjustments or re-operations while preserving task progress. We further introduce the Failure State Recovery Benchmark (FSR-Bench), which evaluates recovery from intermediate failure states under local deviations and structural anomalies. Experiments across multiple VLA backbones, simulation benchmarks, and real-world dual-arm tasks show consistent improvements, with average task-success gains of 14.5 points in simulation and 15.9 points in the real world. Code, models, and data are available at <https://github.com/xiaojunlan/care>

I. INTRODUCTION

Vision-Language-Action (VLA) policies have substantially improved the generality of robotic manipulation [1], [2]. However, most policies are still trained primarily on

successful expert demonstrations and evaluated from nominal initial states [3]–[6]. This leaves an important gap between training and deployment: once execution deviates from the nominal trajectory, the policy may enter states that are rarely represented in its training data. Such deviations are common in physical manipulation, e.g., objects may slip, tip, or be misplaced, and coordinated arms may become unsynchronized. All these deviations can quickly propagate into task failure. Thus, robust manipulation requires not only executing nominal behaviors, but also recovering from the intermediate failure states induced by execution itself.

Existing recovery methods mainly address this problem in two ways. One line of work detects failures and rolls the system back or replans from an earlier state [7], [8]. While effective in some settings, these methods largely treat failures as exceptions to be removed rather than experience that can improve the policy. Another line of work trains recovery behaviors from corrective demonstrations collected around perturbed states [9], [10]. However, generic perturbation strategies do not explicitly model which post-failure deviations are actually induced by policy execution. This motivates a different question: *can execution failures themselves provide structured supervision for learning recovery?*

We answer this question with CARE (Corrective Atomic Robotic Execution), a framework that turns execution experience into corrective supervision and uses it for closed-loop

¹All authors are with Dalian University of Technology, Dalian, China.

*Equal contribution.

[†]Corresponding author.

recovery. CARE first collects failed rollouts and characterizes stage-conditioned geometric deviations, such as translational and rotational errors. These empirical failure distributions are then used to synthesize representative failure states and collect targeted corrective demonstrations. Instead of training only on nominal trajectories or arbitrary perturbations, the VLA is therefore exposed to recovery behaviors around failure states that are grounded in its own execution experience.

Learning corrective behaviors alone is insufficient if the system does not know when and how to invoke them. CARE therefore organizes long-horizon manipulation into atomic stages and uses a physically grounded 3D monitor to track stage-relevant geometric conditions during execution. When a deviation is detected, the system issues either an intra-execution adjustment for local correction or a post-execution re-operation when the current atomic stage must be repeated. The same VLA executor handles both nominal and corrective instructions, allowing recovery without switching to a separate recovery policy.

Evaluating such behavior is also difficult with existing manipulation benchmarks, which primarily measure task success from nominal initial states. We therefore introduce the Failure State Recovery Benchmark (FSR-Bench), which evaluates recovery directly from intermediate failure states. FSR-Bench contains local geometric deviations as well as structural failures that require multi-step correction or co-ordinated multi-arm recovery, and reports recovery success independently from nominal task execution.

We evaluate CARE on RoboTwin 2.0 [6], RoboFactory [5], FSR-Bench, and real-world dual-arm manipulation tasks. Across benchmark–backbone settings, CARE yields an average task-success gain of 14.5 points in simulation and 15.9 points in the real world, together with a 7.5-point gain in recovery success on FSR-Bench. These results demonstrate that execution failures provide effective supervision for learning corrective behaviors in VLA policies.

II. RELATED WORK

A. Robotic Self-correction Systems

Robust manipulation relies on the synergy between error detection and recovery. For detection, early methods use multimodal fusion [11]–[13] or human intervention [14], while recent VLM-based approaches [10], [15]–[18] may lack physical grounding and be prone to misjudgment. World models [7] can incur latency and hallucination, while 3D point cloud-based monitoring [19] improves physical grounding but remains limited to single-arm settings. For recovery, prior methods mainly rely on rollback [7], [8], VLM-based correction [20], or corrective supervision from random perturbations [9], [10] and human interventions [21]. RL-based methods can also learn robust recovery behaviors [22], [23], but real-robot RL may require reward specification, online exploration, and environment resets. Existing approaches, however, do not explicitly model stage-conditioned post-failure deviations to guide corrective data synthesis. CARE models this empirical failure structure to learn and invoke atomic corrections in multi-arm manipulation.

B. Vision-Language-Action Models

Vision-Language-Action models have emerged as a general paradigm for language-conditioned robot manipulation. Existing models mainly generate actions through continuous generative modeling, such as diffusion or flow matching [1], [24]–[28], or autoregressive action tokenization [2], [29]–[32]. While these approaches substantially improve generalization and action modeling, failure recovery remains relatively underexplored. CARE is orthogonal to the underlying VLA architecture and is instantiated on representative continuous and tokenized backbones, π_0 and π_0 -FAST.

C. Robotic Manipulation Benchmarks

Existing benchmarks have advanced from short-horizon single-arm tasks [4], [33], long-horizon multi-tasking and everyday environments [3], [34]–[39], and real-world cross-embodiment generalization [40] to multi-arm collaborative manipulation [5], [6]. However, most benchmarks focus on task execution from predefined initial states and provide limited support for evaluating recovery from intermediate failures. FSR-Bench fills this gap by evaluating self-correction in dual-arm manipulation under execution deviations.

III. METHOD

A. Overview

CARE addresses two complementary aspects of failure recovery: *where recovery supervision should come from* and *when corrective behavior should be invoked*. First, Experience-Guided Corrective Data Synthesis models stage-conditioned deviations observed in failed executions and uses them to generate corrective demonstrations. Second, Atomic Corrective Execution decomposes a task into atomic stages and uses 3D geometric monitoring to select nominal or corrective instructions during execution. Both nominal and corrective instructions are executed by the same VLA policy. Figure 2 illustrates the overall framework.

B. Experience-Guided Corrective Data Synthesis

1) *Failure Modeling*: We begin by executing nominal atomic plans without corrective supervision and collect failure cases from 100 rollouts in simulation and the real world. A failure is recorded when an atomic stage does not satisfy its termination condition.

For a failure occurring at stage k_i , we characterize the deviation at the corresponding stage-critical event as

$$d_i = (\Delta\tau_i, \Delta R_i), \quad (1)$$

where $\Delta\tau_i \in \mathbb{R}^3$ and ΔR_i denote the relative translational and rotational deviations, respectively. We model rotational deviations as yaw offsets Δr_i , with $\Delta R_i = R_z(\Delta r_i)$, where $R_z(\cdot)$ denotes rotation about the vertical axis. The deviations are measured in relative coordinates: for grasping, they describe object–gripper misalignment at gripper closing; for placement, they describe object–target misalignment at gripper opening.

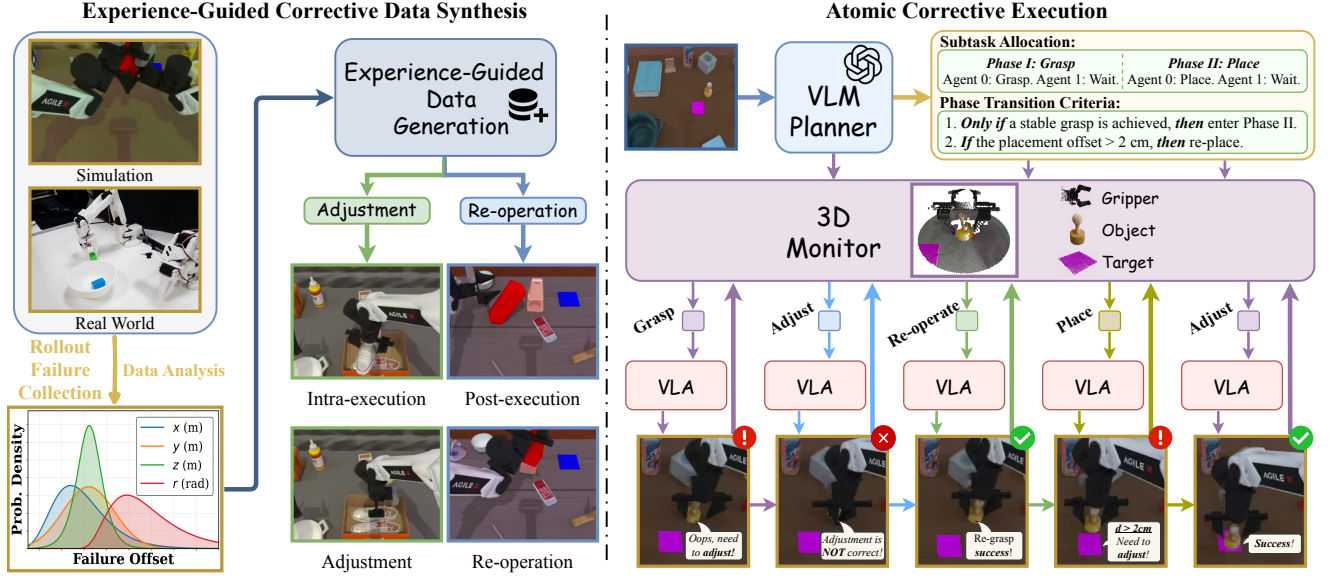


Fig. 2. Overview of CARE. CARE first models stage-conditioned deviations from failed executions and uses them to generate corrective demonstrations. At inference time, stage-wise planning and 3D monitoring select nominal or corrective atomic instructions for a unified VLA executor.

Because different stages exhibit different failure patterns, we model a stage-conditioned deviation distribution

$$p(d | k). \quad (2)$$

For each task and atomic stage, we independently fit each scalar component ($\Delta x, \Delta y, \Delta z, \Delta r$) to Gaussian, Beta, Gamma, Weibull, Log-Normal, and Uniform candidate distributions, and select the best fit using AIC [41] and the KS test [42].

2) *Corrective Data Synthesis*: To generate a new failure state at stage k , we sample

$$d = (\Delta\tau, \Delta R) \sim p(d | k), \quad (3)$$

and perturb the stage-relevant relative pose:

$$\tau_k \leftarrow \tau_k + \Delta\tau, \quad R_k \leftarrow \Delta R R_k. \quad (4)$$

The system is then rolled forward under physical dynamics. Thus, synthesized failures are newly generated physical states rather than direct replays of previously collected failures. In simulation, an IK/motion-planning oracle generates corrective demonstrations from these states, whereas real-world demonstrations are collected through human teleoperation. Both consist of short corrective segments that terminate once the atomic recovery target is reached.

C. Atomic Corrective Execution Framework

Learning corrective behaviors alone does not determine when they should be invoked. CARE therefore performs recovery at the atomic-skill level. A VLM planner decomposes the task and specifies stage-level recovery semantics, while a 3D geometric monitor determines whether to continue the current stage, trigger adjustment, or trigger re-operation. The planner and monitor only select the atomic instruction; all low-level actions, including corrective actions, are generated by the same VLA executor.

1) *VLM Planner*: We use GPT-4.1 (temperature 0) to decompose the high-level instruction ℓ and initial global-view image I_0^g into K atomic stages. For each stage k , it produces

$$\Pi_k = (u_k, \gamma_k, \psi_k, \mathcal{U}_k), \quad (5)$$

where u_k is the nominal atomic instruction, γ_k specifies the geometric cues relevant to recovery, ψ_k is the stage-termination predicate, and

$$\mathcal{U}_k = \mathcal{U}_k^{\text{adj}} \cup \mathcal{U}_k^{\text{re}} \quad (6)$$

contains instructions for intra-execution adjustment and post-execution re-operation.

The planner is queried once at task initialization. The resulting stage plans and candidate instructions remain fixed, while CARE updates the active stage k_t and selects the executed instruction online. This separates semantic task decomposition from high-frequency visuomotor control and avoids repeated VLM calls during recovery.

2) *3D Geometric Monitor*: The 3D monitor is a point-cloud predicate checker rather than a learned recovery policy or a VLM-based judge. It continuously tracks gripper open/close states and invokes geometric perception at stage-critical transitions, such as grasp closure or object release, or shortly before a predicted transition when intra-execution adjustment is enabled.

For each triggered check, SAM 3 [43] extracts masks for the gripper, manipulated object, and target region from the RGB observations, and Depth Anything 3 [44] provides depth estimates. These observations are fused into the gripper, object, and target point clouds $\mathcal{P}_{g,t}$, $\mathcal{P}_{o,t}$, and $\mathcal{P}_{\text{tar},t}$. The monitor then computes a compact geometric state

$$q_t = \Phi(\mathcal{P}_{g,t}, \mathcal{P}_{o,t}, \mathcal{P}_{\text{tar},t}), \quad (7)$$

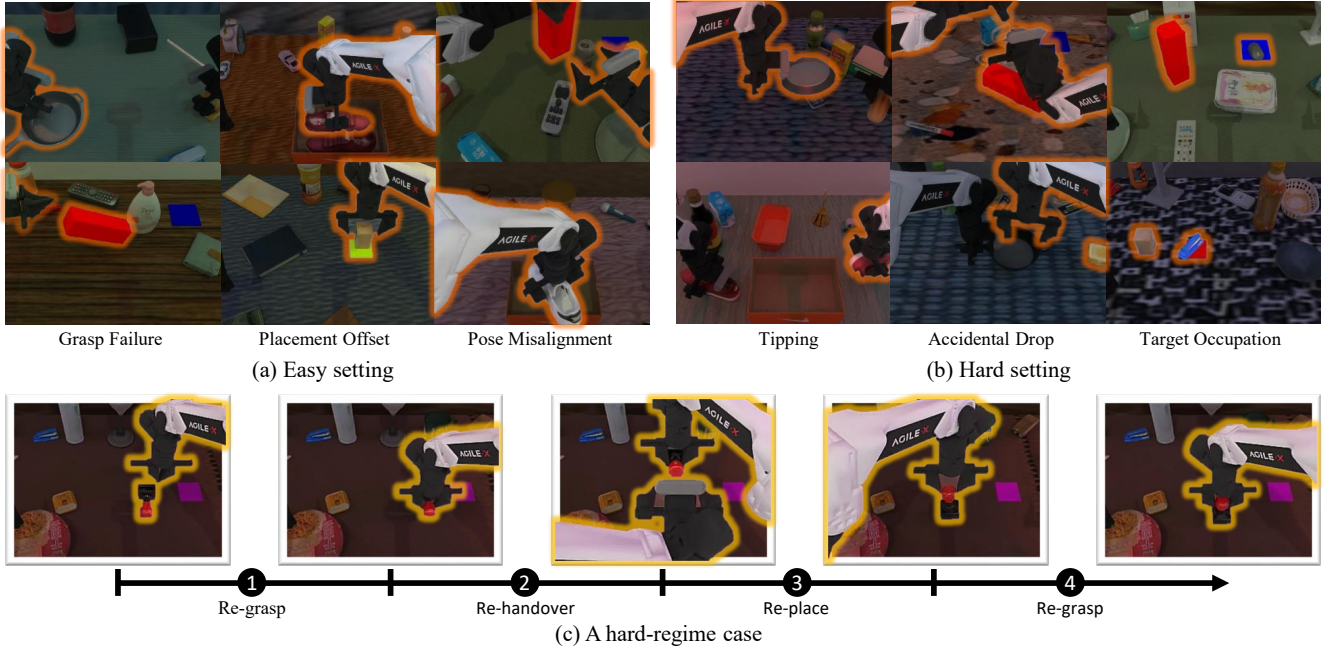


Fig. 3. **FSR-Bench**. Recovery scenarios are grouped into two regimes by corrective complexity: (a) **Easy**, including local deviations that typically admit single-step correction, such as *Grasp Failure*, *Placement Offset*, and *Pose Misalignment*; and (b) **Hard**, including structural anomalies that require multi-step recovery or multi-arm coordination, such as *Tipping*, *Accidental Drop*, and *Target Occupation*. Each column shows two representative initializations. (c) **A hard-regime case**, where repeated monitor–execute–verify transitions select a coordinated sequence of bimanual corrective skills online.

with stage-relevant relations such as gripper–object alignment, object–target displacement, and post-contact stability.

The monitor uses skill-level geometric predicates shared across tasks composed of known atomic skills, with numerical tolerances calibrated from rollout statistics. A new atomic skill requires defining its predicate template once, which can then be reused across tasks.

Conditioned on the active plan Π_{k_t} and geometric state q_t , the monitor selects

$$\tilde{u}_t = f_{\text{mon}}(\Pi_{k_t}, q_t), \quad \tilde{u}_t \in \{u_{k_t}\} \cup \mathcal{U}_{k_t}^{\text{adj}} \cup \mathcal{U}_{k_t}^{\text{re}}. \quad (8)$$

Consistent geometry retains u_{k_t} ; in-stage deviations trigger adjustment, while failed outcomes trigger re-operation. After each atomic execution, the monitor reapplies Eq. (8): CARE advances to $k_t + 1$ when ψ_{k_t} holds; otherwise, it retains k_t and retries until the retry budget is exhausted, after which recovery fails. Repeated monitor–execute–verify transitions thereby generate multi-step recovery over the fixed plan without a predefined sequence.

3) *VLA Executor*: The VLA executor receives the current observation o_t together with the selected atomic instruction:

$$a_t \sim \pi(a_t \mid o_t, \tilde{u}_t). \quad (9)$$

Here, o_t contains the robot state together with the global and wrist-view RGB observations. The same VLA policy executes nominal, adjustment, and re-operation instructions; CARE therefore invokes learned corrective behaviors without switching to a separate recovery policy.

D. FSR-Bench

FSR-Bench evaluates failure recovery independently of nominal task execution. Instead of starting from clean initial states, each episode begins from an intermediate failure state and requires the policy to restore a task-feasible state. The benchmark contains 36 recovery scenarios across five tasks, organized into two regimes by corrective complexity. *Easy* contains 21 local failures that typically admit a single corrective operation, such as grasp failure, placement offset, and pose misalignment. *Hard* contains 15 structural failures that require multi-step correction or coordinated multi-arm recovery, such as tipping, accidental drop, and target occupation. Representative scenarios are shown in Fig. 3.

a) *Training and Test Distributions*: To decouple recovery training from the benchmark test distribution, the recovery training data used in FSR-Bench are collected by uniformly sampling perturbations within the predefined perturbation range of each recovery scenario.

In contrast, benchmark evaluation reflects failures arising during actual policy execution. We collect failed rollouts from a mixture of nominal policies without recovery and estimate a stage-conditioned empirical failure distribution

$$p_{\text{fail}}(d \mid k), \quad (10)$$

where d denotes the geometric deviation at atomic stage k .

At evaluation time, we sample

$$d \sim p_{\text{fail}}(d \mid k), \quad (11)$$

inject the sampled deviation into the corresponding stage of a nominal execution, and roll the system forward under

physical dynamics. The resulting state, rather than the injected pose itself, is used as the recovery initialization. Each evaluation episode contains a new failure state rather than a replay of a collected failure. All baseline and CARE variants use the same uniformly sampled corrective training data and evaluation initializations. Thus, their comparison evaluates atomic corrective execution under failures generated from a mixture of nominal policies.

Evaluation episodes additionally include variations in object geometry, table texture, lighting, distractor objects, and object placement. For each scenario, all methods are evaluated on the same pre-generated failure initializations.

b) Evaluation Protocol: Each method is evaluated over $N_{\text{eval}} = 100$ randomized trials per task–regime pair, with initializations drawn from the corresponding recovery scenarios. Recovery Success Rate (RSR) is defined as

$$\text{RSR} = \frac{1}{N_{\text{eval}}} \sum_{j=1}^{N_{\text{eval}}} \mathbb{I}[r_j = 1], \quad (12)$$

where $r_j = 1$ if the scenario-specific recovery target is reached within its prescribed horizon. For each scenario, its target and horizon are shared across methods. For each task and regime, RSR is computed over 100 trials sampled from the corresponding constituent failure states.

IV. EXPERIMENTS

A. Experimental Setup

a) Tasks and baselines: We evaluate CARE on seven hard bimanual tasks from RoboTwin 2.0 [6], three multi-arm tasks from RoboFactory [5], five FSR-Bench recovery tasks, and four real-world dual-arm tasks. For RoboTwin 2.0, we use the `demo_randomized` setting for training and evaluation. We instantiate CARE on π_0 [1] and π_0 -FAST [2], and compare against their vanilla counterparts, ACT [45], RDT-1B [24], DP [46], and DP3 [47]. On FSR-Bench, we additionally instantiate CARE on RDT-1B.

b) Training protocol: For standard simulation tasks, each method uses 150 nominal expert demonstrations and 50 additional trajectories per error type. CARE uses experience-guided corrective trajectories, while baselines use an equal number of nominal atomic-stage trajectories. The corrective data are synthesized from stage-conditioned failure distributions estimated from 100 preliminary rollouts. Policies are trained for 40,000 gradient steps with a batch size of 32 and an action horizon of 50 on two NVIDIA A800 SXM4 80GB GPUs. For FSR-Bench, all variants use 50 uniformly sampled corrective trajectories per recovery type and are trained for 20,000 steps with the same batch size and horizon.

c) Real-world setup and evaluation: We evaluate four bimanual tasks on a dual-arm LeRobot SO-101 platform with 12 DoF (6 per arm), one global RGB camera, and one wrist RGB camera per arm, all operating at 30 Hz. For each task, we collect 50 nominal demonstrations and 20 additional trajectories per error type, following the same matched-data protocol as in simulation. We use 100 preliminary executions per task to estimate the empirical failure distribution. Real-world task success is evaluated over 100 trials per task.

TABLE I

PERFORMANCE ON BIMANUAL TASKS IN ROBOTWIN 2.0 (HARD) [6].
SHORT/LONG DENOTE SHORT-HORIZON (<200 STEPS) AND LONG-HORIZON (200–400 STEPS). LP: LIFT POT; SS: STAMP SEAL; PBS: PLACE BREAD SKILLET; PDS: PLACE DUAL SHOES; POC: PUT OBJECT CABINET; HB: HANDOVER BLOCK; DBB: DUMP BIN BIGBIN.

Model	Short			Long				Avg.
	LP	SS	PBS	PDS	POC	HB	DBB	
ACT [45]	46%	0%	5%	0%	23%	0%	74%	21.1%
RDT-1B [24]	15%	2%	2%	2%	23%	7%	75%	18.0%
DP [46]	52%	0%	6%	4%	42%	21%	53%	25.4%
DP3 [47]	27%	1%	9%	7%	52%	14%	87%	28.1%
π_0 -FAST [2]	18%	2%	2%	10%	23%	24%	43%	17.4%
π_0 -FAST (CARE)	40%	17%	28%	33%	32%	16%	66%	33.1%
π_0 [1]	57%	3%	28%	14%	59%	27%	89%	39.6%
π_0 (CARE)	91%	29%	41%	40%	68%	62%	94%	60.7%

TABLE II

PERFORMANCE ON MULTI-ARM COLLABORATION TASKS IN ROBOFACTORY [5].

Model	Place food (Two Arms)	Stack cubes three (Three Arms)	Take photo (Four Arms)	Avg.
DP [46]	20%	22%	20%	20.7%
π_0 -FAST [2]	27%	9%	44%	26.7%
π_0 -FAST (CARE)	40%	17%	51%	36.0%
π_0 [1]	56%	48%	82%	62.0%
π_0 (CARE)	73%	61%	88%	74.0%

B. Main Results in Simulation

Table I evaluates CARE on hard bimanual tasks in RoboTwin 2.0, where recovery requires precise geometric correction under challenging interactions. CARE substantially improves both VLA backbones, raising average success rates from 39.6% to 60.7% (+21.1 points) for π_0 and from 17.4% to 33.1% (+15.7 points) for π_0 -FAST. These gains span short- and long-horizon tasks. Table II further shows CARE’s benefits for multi-arm collaboration in RoboFactory. Across two-, three-, and four-arm tasks, CARE improves both backbones, with average absolute gains of +9.3 points for π_0 -FAST and +12.0 points for π_0 . These results demonstrate CARE’s failure-recovery capability across task horizons and multi-arm settings.

C. Results in FSR-Bench

Table III shows that CARE’s Atomic Corrective Execution consistently improves recovery across all three VLA backbones and both regimes. To isolate execution, experience-guided synthesis is disabled for this benchmark, and each vanilla/CARE pair uses the same 50 uniformly sampled corrective atomic trajectories per recovery type. On average, RDT-1B improves from 21.8% to 25.2% in the Easy regime and from 2.6% to 6.0% in the Hard regime. π_0 -FAST improves from 24.4% to 32.6% in the Easy regime and from 4.6% to 10.8% in the Hard regime. The largest gains are achieved by π_0 , whose average RSR rises from 38.8% to 57.2% in the Easy regime and from 15.8% to 21.2% in the

TABLE III
PERFORMANCE COMPARISON ON FSR-BENCH. DSR: DUAL SHOES RECOVERY; SSR: SEAL STAMP RECOVERY; SPR: SKILLET PLACEMENT RECOVERY; BHR: BLOCK HANDOVER RECOVERY; PLR: POT LIFT RECOVERY.

Task	RDT-1B [24]		RDT-1B (CARE)		π_0 -FAST [2]		π_0 -FAST (CARE)		π_0 [1]		π_0 (CARE)	
	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard
DSR	42%	2%	51%	4%	44%	2%	48%	11%	63%	7%	69%	9%
SSR	10%	2%	11%	2%	27%	5%	30%	9%	36%	18%	54%	24%
SPR	11%	2%	13%	14%	12%	6%	16%	12%	25%	32%	31%	40%
BHR	36%	0%	39%	2%	35%	2%	45%	4%	40%	11%	54%	18%
PLR	10%	7%	12%	8%	4%	8%	24%	18%	30%	11%	78%	15%
Avg.	21.8%	2.6%	25.2%	6.0%	24.4%	4.6%	32.6%	10.8%	38.8%	15.8%	57.2%	21.2%

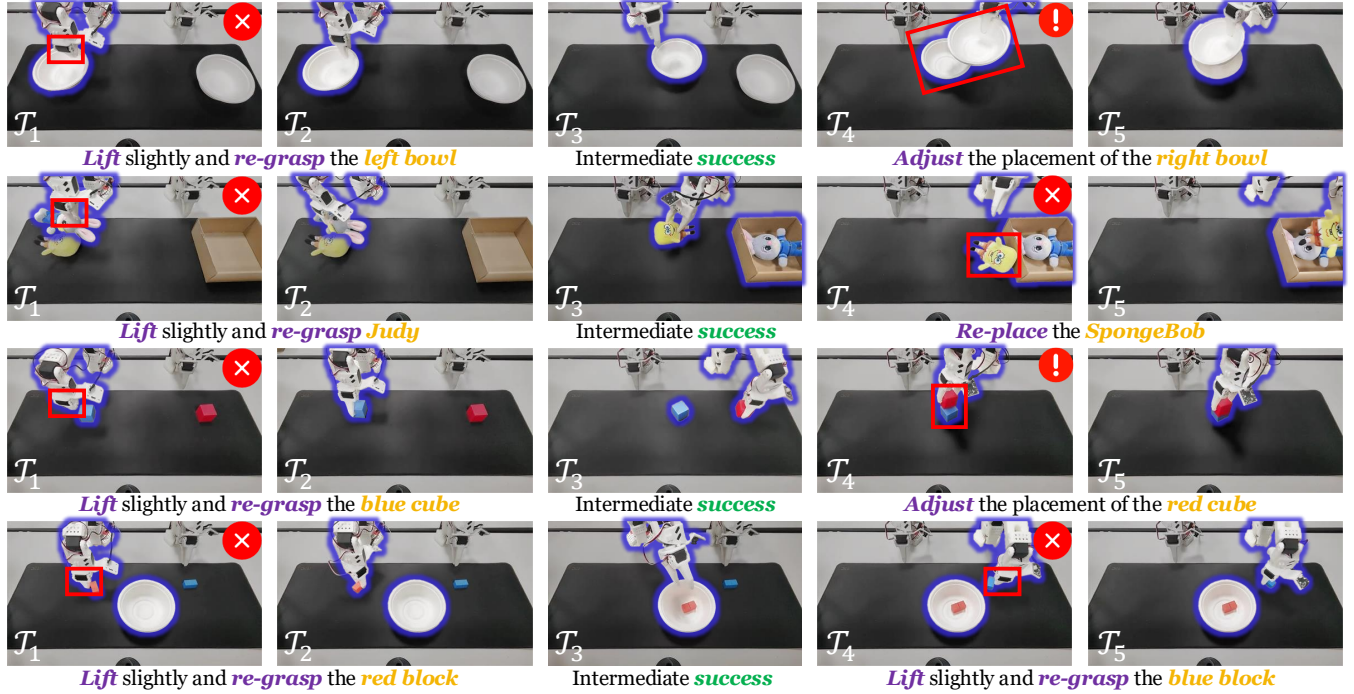


Fig. 4. Qualitative results on four real-world dual-arm SO-101 tasks. From top to bottom: Stack Two Bowls, Pass Two Toys, Stack Two Cubes, and Place Two Cubes. **✗**: Requiring post-execution re-operation; **!**: Requiring intra-execution adjustment.

Hard regime. The consistent gains across backbones suggest that the execution mechanism is not model-specific. Overall, matched corrective supervision alone is insufficient for robust recovery, which also requires atomic corrective execution.

D. Main Results in Real World

We validate CARE on a real-world SO-101 dual-arm robot across four long-horizon manipulation tasks (Fig. 4). For system-level comparison against a dedicated recovery method, we reproduce the complete FailSafe [9] pipeline on the same platform with identical VLA backbones and evaluation protocol. Table IV reports success rates over 100 trials per task. CARE improves the average success rate of π_0 from 30.8% to 50.0% and that of π_0 -FAST from 18.8% to 31.3%, outperforming the dedicated FailSafe recovery baseline by 13.7 and 7.3 points, respectively.

V. ABLATION STUDY

We conduct ablation studies on selected simulation tasks from RoboTwin 2.0 [6] and RoboFactory [5], as well as

TABLE IV
SUCCESS RATES ON SO-101 DUAL-ARM ROBOT TASKS.

Task	π_0 [1]			π_0 -FAST [2]		
	Base	FailSafe	CARE	Base	FailSafe	CARE
Stack Two Cubes	2%	6%	14%	0%	2%	6%
Stack Two Bowls	44%	48%	62%	27%	36%	46%
Pass Two Toys	15%	21%	42%	7%	11%	17%
Place Two Cubes	62%	70%	82%	41%	47%	56%
Avg.	30.8%	36.3%	50.0%	18.8%	24.0%	31.3%

real-world tasks on the dual-arm SO-101 platform, following the corresponding experimental configurations in Sec. IV-A. In Sec. V-A, for each error type, the **Data** setting replaces the 50 additional nominal atomic-stage trajectories with 50 corrective trajectories, keeping the amount of added training data fixed. In Sec. V-B, both the **Uniform Random** and **Experience-Guided** strategies use 50 corrective trajectories per error type under identical training settings.

TABLE V

ABLATION OF CARE COMPONENTS. PF: PLACE FOOD; PDS: PLACE DUAL SHOES; LP: LIFT POT; PBS: PLACE BREAD SKILLET; SS: STAMP SEAL.

Model	Method		Task					
	Execution	Data	PF	PDS	LP	PBS	SS	Avg.
π_0 [1]	–	–	56%	14%	57%	28%	3%	31.6%
	✓	–	58%	22%	58%	30%	11%	35.8%
	–	✓	61%	33%	62%	26%	22%	40.8%
	✓	✓	73%	40%	91%	41%	29%	54.8%
π_0 -FAST [2]	–	–	27%	10%	18%	2%	2%	11.8%
	✓	–	30%	11%	18%	6%	4%	13.8%
	–	✓	32%	31%	23%	6%	12%	20.8%
	✓	✓	40%	33%	40%	28%	17%	31.6%

A. Ablation of Components

Table V ablates CARE’s two core components: corrective training data (**Data**) and the atomic corrective execution framework (**Execution**). For both backbones, **Data** provides the larger gain. On π_0 , **Data** alone improves the average success rate from 31.6% to 40.8% (+9.2 points), while **Execution** alone raises it to 35.8% (+4.2 points). A similar trend holds for π_0 -FAST, where **Data** alone improves performance from 11.8% to 20.8% (+9.0 points), whereas **Execution** alone raises it to 13.8% (+2.0 points). Full CARE achieves the best results, outperforming the respective baselines by 23.2 points on π_0 and 19.8 points on π_0 -FAST. Because full CARE and the data-only variant use identical nominal and corrective training data, their 14.0- and 10.8-point gaps isolate the additional benefit of online atomic corrective execution beyond corrective supervision alone.

B. Ablation of Corrective Data Collection

TABLE VI

ABLATION OF CORRECTIVE DATA COLLECTION. SCT: STACK CUBES THREE; PDS: PLACE DUAL SHOES; LP: LIFT POT; SS: STAMP SEAL; HB: HANDOVER BLOCK.

Task	Uniform Random		Experience-Guided (Ours)	
	π_0 (CARE)	π_0 -FAST (CARE)	π_0 (CARE)	π_0 -FAST (CARE)
SCT	53%	5%	61%	17%
PDS	27%	24%	40%	33%
LP	66%	27%	91%	40%
SS	20%	7%	29%	17%
HB	56%	16%	62%	16%
Avg.	44.4%	15.8%	56.6% (+12.2 pts)	24.6% (+8.8 pts)

To isolate the effect of the sampling distribution, we compare our Experience-Guided Corrective Data Synthesis with a FailSafe-style Uniform Random strategy [9]. Both use identical deviation dimensions and ranges, numbers of corrective trajectories, and training settings; only the sampling distribution differs: uniform sampling versus the fitted $p(d | k)$. As shown in Table VI, experience-guided sampling yields consistently higher average performance than uniform sampling, improving the average success rate by 12.2 points for π_0 (CARE) and 8.8 points for π_0 -FAST (CARE). These results suggest that uniformly sampled perturbations often miss physically meaningful long-tail failure modes, whereas

TABLE VII

ABLATION OF MONITORING MECHANISMS ON π_0 . TASKS MARKED WITH † ARE REAL-WORLD.

Task	Monitoring Accuracy ↑		Latency (s/call) ↓	
	VLM	3D Monitor (Ours)	VLM	3D Monitor (Ours)
Lift Pot (LP)	80%	98%	2.32	0.24
Place Dual Shoes (PDS)	85%	95%	2.51	0.25
Handover Block (HB)	50%	96%	4.53	0.24
Pass Two Toys (PTT)†	70%	86%	3.77	0.24
Stack Two Bowls (STB)†	72%	94%	3.89	0.24
Avg.	71.4%	93.8% (+22.4 pts)	3.40	0.24 (14.1× faster)

empirically grounded synthesis provides more effective recovery supervision for closed-loop correction.

C. Ablation of Monitoring Mechanisms

We replace our 3D monitor with a GPT-4.1-based VLM monitor and evaluate both on the same triggered observations with manual decision labels. As shown in Table VII, across three simulated and two real-world tasks, our monitor improves average accuracy from 71.4% to 93.8% (+22.4 points), while reducing end-to-end monitor latency, including perception, point-cloud construction, and predicate evaluation, from 3.40 s to 0.24 s (14.1× faster). These results demonstrate more accurate and efficient monitoring across simulation and real-world deployment.

VI. CONCLUSION

We present CARE, a framework for improving failure recovery in long-horizon multi-arm VLA manipulation. CARE treats execution failures as structured supervision, combining Experience-Guided Corrective Data Synthesis with 3D-monitor-driven atomic corrective execution. We introduce FSR-Bench to evaluate recovery from empirically grounded intermediate failure states. Across simulation, recovery-specific, and real-world evaluations, CARE consistently improves recovery and robustness over diverse VLA backbones. These results suggest that robust embodied intelligence requires both strong nominal execution and effective grounded recovery from failures. Future work will integrate reinforcement learning into CARE to further enhance recovery.

REFERENCES

- [1] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “pi_0: A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [2] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, “Fast: Efficient action tokenization for vision-language-action models,” *arXiv preprint arXiv:2501.09747*, 2025.
- [3] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Liber0: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [4] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, “Rlbench: The robot learning benchmark & learning environment,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [5] Y. Qin, L. Kang, X. Song, Z. Yin, X. Liu, X. Liu, R. Zhang, and L. Bai, “Robofactory: Exploring embodied agent collaboration with compositional constraints,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 10 075–10 085.

- [6] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu *et al.*, “Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” *arXiv preprint arXiv:2506.18088*, 2025.
- [7] S. Pan, Y. Xu, R. Xu, Z. Zhou, S. Wu, and Z. Yu, “Self-correcting robot manipulation via gaussian-splatted foresight,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 25, 2025, pp. 26 642–26 650.
- [8] S. Xu, Z. Wang, Y. Wang, C. Xia, T. Huang, and C. Xu, “Affordance field intervention: Enabling vlas to escape memory traps in robotic manipulation,” *arXiv preprint arXiv:2512.07472*, 2025.
- [9] Z. Lin, J. Duan, H. Fang, D. Fox, R. Krishna, C. Tan, and B. Wen, “Failsafe: Reasoning and recovery from failures in vision-language-action models,” *arXiv preprint arXiv:2510.01642*, 2025.
- [10] Y. Dai, J. Lee, N. Fazeli, and J. Chai, “Racer: Rich language-guided failure recovery policies for imitation learning,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 15 657–15 664.
- [11] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina, “Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies,” *arXiv preprint arXiv:2503.08558*, 2025.
- [12] Q. Bu, J. Zeng, L. Chen, Y. Yang, G. Zhou, J. Yan, P. Luo, H. Cui, Y. Ma, and H. Li, “Closed-loop visuomotor control with generative expectation for robotic manipulation,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 139 002–139 029, 2024.
- [13] J. Li, S. Dong, and E. Adelson, “Slip detection with combined tactile and visual information,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 7772–7777.
- [14] L. X. Shi, Z. Hu, T. Z. Zhao, A. Sharma, K. Pertsch, J. Luo, S. Levine, and C. Finn, “Yell at your robot: Improving on-the-fly from language corrections,” *arXiv preprint arXiv:2403.12910*, 2024.
- [15] W. Xia, R. Feng, D. Wang, and D. Hu, “Phoenix: A motion-based self-reflection framework for fine-grained robotic action correction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 6981–6990.
- [16] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, “Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation,” *arXiv preprint arXiv:2410.00371*, 2024.
- [17] C. Qi, X. Wang, S. Yong, S. Sheng, H. Mao, S. Srinivasan, M. Nambi, A. Zhang, and Y. Dattatreya, “Self-refining vision language model for robotic failure detection and reasoning,” *arXiv preprint arXiv:2602.12405*, 2026.
- [18] M. Pan, J. Zhang, T. Wu, Y. Zhao, W. Gao, and H. Dong, “Omnimanip: Towards general robotic manipulation via object-centric interaction primitives as spatial constraints,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 17 359–17 369.
- [19] E. Zhou, Q. Su, C. Chi, Z. Zhang, Z. Wang, T. Huang, L. Sheng, and H. Wang, “Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 6919–6929.
- [20] Y. Yang, Z. Duan, T. Xie, F. Cao, P. Shen, P. Song, P. Jin, G. Sun, S. Xu, Y. You *et al.*, “Fpc-vla: A vision-language-action framework with a supervisor for failure prediction and correction,” *arXiv preprint arXiv:2509.04018*, 2025.
- [21] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo *et al.*, “ $\pi_{0.6}^*$: a VLA That Learns From Experience,” *arXiv preprint arXiv:2511.14759*, 2025.
- [22] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, “Serl: A software suite for sample-efficient robotic reinforcement learning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 16 961–16 969.
- [23] S. Vats, M. Likhachev, and O. Kroemer, “Efficient recovery learning using model predictive meta-reasoning,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 7258–7264.
- [24] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “Rdt-1b: a diffusion foundation model for bimanual manipulation,” *arXiv preprint arXiv:2410.07864*, 2024.
- [25] J. Wen, Y. Zhu, J. Li, Z. Tang, C. Shen, and F. Feng, “Dexvla: Vision-language model with plug-in diffusion expert for general robot control,” *arXiv preprint arXiv:2502.05855*, 2025.
- [26] J. Liu, H. Chen, P. An, Z. Liu, R. Zhang, C. Gu, X. Li, Z. Guo, S. Chen, M. Liu *et al.*, “Hybridvla: Collaborative diffusion and autoregression in a unified vision-language-action model,” *arXiv preprint arXiv:2503.10631*, 2025.
- [27] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [28] Z. Zhang, J. Xiao, Z. Zhang, Y. Wang, L. Kang, Y. Qin, C. Xia, H. Zhou, T. Fu, E. Zhou *et al.*, “Ma-vla: Multi-arm vision-language-action model for collaboration and compositional generalization,” *arXiv preprint arXiv:2608.25864*, 2026.
- [29] Y. Liu, S. Zhang, Z. Dong, B. Ye, T. Yuan, X. Yu, L. Yin, C. Lu, J. Shi, L. J.-T. Yu *et al.*, “Faster: Toward efficient autoregressive vision language action modeling via neural action tokenization,” *arXiv preprint arXiv:2512.04952*, 2025.
- [30] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li, “Univla: Learning to act anywhere with task-centric latent actions,” *arXiv preprint arXiv:2505.06111*, 2025.
- [31] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn *et al.*, “Cot-vla: Visual chain-of-thought reasoning for vision-language-action models,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 1702–1713.
- [32] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “Open-vla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [33] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning,” in *Conference on robot learning*. PMLR, 2020, pp. 1094–1100.
- [34] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [35] T. Mu, Z. Ling, F. Xiang, D. Yang, X. Li, S. Tao, Z. Huang, Z. Jia, and H. Su, “Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations,” *arXiv preprint arXiv:2107.14483*, 2021.
- [36] S. Zhang, Z. Xu, P. Liu, X. Yu, Y. Li, Q. Gao, Z. Fei, Z. Yin, Z. Wu, Y.-G. Jiang *et al.*, “Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 11 142–11 152.
- [37] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, “Robocasa: Large-scale simulation of everyday tasks for generalist robots,” *arXiv preprint arXiv:2406.02523*, 2024.
- [38] C. Li, R. Zhang, J. Wong, C. Gokmen, S. Srivastava, R. Martín-Martín, C. Wang, G. Levine, M. Lingelbach, J. Sun *et al.*, “Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation,” in *Conference on Robot Learning*. PMLR, 2023, pp. 80–93.
- [39] B. Shen, F. Xia, C. Li, R. Martín-Martín, L. Fan, G. Wang, C. Pérez-D’Arpino, S. Buch, S. Srivastava, L. Tchammi *et al.*, “igibson 1.0: A simulation environment for interactive tasks in large realistic scenes,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 7520–7527.
- [40] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain *et al.*, “Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6892–6903.
- [41] H. Akaike, “A new look at the statistical model identification,” *IEEE transactions on automatic control*, vol. 19, no. 6, pp. 716–723, 1974.
- [42] F. J. Massey Jr, “The kolmogorov-smirnov test for goodness of fit,” *Journal of the American statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.
- [43] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang *et al.*, “Sam 3: Segment anything with concepts,” *arXiv preprint arXiv:2511.16719*, 2025.
- [44] H. Lin, S. Chen, J. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang, “Depth anything 3: Recovering the visual space from any views,” *arXiv preprint arXiv:2511.10647*, 2025.

- [45] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023.
- [46] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.
- [47] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations,” *arXiv preprint arXiv:2403.03954*, 2024.

Supplementary Material

VII. OVERVIEW

This supplement provides implementation details, benchmark specifications, additional analyses, training protocols, and qualitative results that complement the main paper. It is organized as follows:

- **Sec. VIII: Experience-Guided Corrective Data Synthesis Details.** Additional details on stage-conditioned failure modeling, distribution fitting, and corrective-state synthesis.
- **Sec. IX: Atomic Corrective Execution Details.** Implementation details of the offline VLM planner, event-triggered 3D monitor, and geometric predicates.
- **Sec. X: FSR-Bench Details.** Recovery horizons, failure-state taxonomy, train/test distribution construction, and evaluation protocol.
- **Sec. XI: Additional Experimental Analysis.** Additional analysis of corrective interventions, the number of experience-guided rollouts, rollback recovery, and monitor accuracy.
- **Sec. XII: Training and Evaluation Protocols.** Training hyperparameters, data budgets, deployment details, and evaluation settings.
- **Sec. XIII: Visualization.** Qualitative visualizations of nominal and corrective execution for *Place Dual Shoes*, together with all 36 FSR-Bench failure states.

VIII. EXPERIENCE-GUIDED CORRECTIVE DATA SYNTHESIS DETAILS

As described in the main paper, CARE begins by executing the nominal atomic plans without corrective supervision and collecting failures from 100 preliminary rollouts. A failure is recorded when an atomic stage does not satisfy its termination condition. For a failure at stage k_i , we record the stage-critical relative deviation

$$d_i = (\Delta\tau_i, \Delta R_i), \quad (13)$$

where $\Delta\tau_i \in \mathbb{R}^3$ is the translational deviation and ΔR_i is the rotational deviation. For grasping, the deviation is measured between the gripper and the manipulated object at gripper closure. For placement, it is measured between the object and the target at gripper opening. We therefore model a stage-conditioned failure prior $p(d \mid k)$ rather than a single task-level perturbation distribution.

For implementation, we parameterize the translational component by (dx, dy, dz) . In the tasks used in our experiments, the dominant rotational variation around grasping and placement is yaw, so the rotational component is represented by a scalar dr in the distribution-fitting analysis. This is the four-variable representation used in the rollout-stability experiments in Sec. XI-C.

For each scalar deviation variable, we consider a small family of parametric distributions, including Gaussian, Beta,

Gamma, Weibull, Log-Normal, and Uniform distributions. We select the best-fitting family using goodness-of-fit statistics together with the Akaike Information Criterion (AIC). This procedure keeps the corrective-data prior grounded in failures encountered by the policy instead of assuming a manually chosen perturbation shape.

To synthesize a corrective trajectory at stage k , we sample

$$d = (\Delta\tau, \Delta R) \sim p(d \mid k), \quad (14)$$

perturb the stage-relevant relative pose,

$$\tau_k \leftarrow \tau_k + \Delta\tau, \quad R_k \leftarrow \Delta R R_k, \quad (15)$$

and then roll the system forward under the simulator or real-world physical dynamics. The resulting state, rather than the injected pose itself, is used as the start state for expert correction. Consequently, synthesized failures are new physical states generated from the learned failure prior rather than direct replays of previously collected failures.

Corrective demonstrations are collected only for the recovery behavior needed at the current atomic stage. For example, a re-grasp trajectory terminates once a stable grasp has been recovered instead of continuing to the end of the full task. This short-horizon design reduces corrective-data collection cost while preserving the supervision required to learn the corrective atomic behavior.

a) *Relation to FSR-Bench.* The experience-guided distribution above is used for corrective data synthesis in the standard RoboTwin, RoboFactory, and real-world CARE experiments. FSR-Bench uses a deliberately different protocol to avoid coupling recovery training to the test distribution: its recovery training trajectories are collected from *uniformly sampled* perturbations within the predefined range of each recovery scenario, whereas evaluation initializations are sampled from empirical stage-conditioned failure distributions estimated from nominal-policy rollouts. We detail this separation in Sec. X.

IX. ATOMIC CORRECTIVE EXECUTION DETAILS

A. Offline VLM planner

The planner is implemented with GPT-4.1 at temperature 0.0 and is queried once at task initialization. Given the high-level instruction and the initial global-view image, it decomposes the task into K atomic stages. Consistent with the notation of the main paper, each stage plan is

$$\Pi_k = (u_k, \gamma_k, \psi_k, \mathcal{U}_k), \quad \mathcal{U}_k = \mathcal{U}_k^{\text{adj}} \cup \mathcal{U}_k^{\text{re}}, \quad (16)$$

where u_k is the nominal atomic instruction, γ_k specifies the geometric cues used by the monitor, ψ_k is the stage-termination condition, and $\mathcal{U}_k^{\text{adj}}$ and $\mathcal{U}_k^{\text{re}}$ contain intra-stage adjustment and stage-level re-operation instructions, respectively. The resulting stage plans remain fixed during low-level execution.

Prompt Template for the Offline VLM Planner

You are an offline VLM planner for CARE (Corrective Atomic Robotic Execution) in multi-arm manipulation.

Your task is to decompose a high-level manipulation instruction into ordered atomic stages and produce the stage plan used by the 3D monitor and VLA executor.

Inputs:

- {global image}
- {task instruction}
- {task description}
- {agent description}
- {atomic action pool}
- {deviation statistics}

Requirements:

1. Decompose the task into ordered atomic stages.
2. For each stage, assign exactly one nominal atomic instruction to each agent.
3. For each stage, provide geometric guidance γ_k describing the task-relevant gripper/object/target relations that should be monitored.
4. For each stage, provide a concise observable termination condition ψ_k .
5. Allocate corrective instructions only for deviation cases relevant to the stage.
6. Separate corrective instructions into:
 - U_k_adj: intra-stage adjustments for local deviations;
 - U_k_re: stage-level re-operations such as re-grasp or re-place.
7. Use only actions from the atomic action pool.
8. Use "Wait for the other robots." for idle agents.
9. Ensure physical feasibility, spatial consistency, and coordinated multi-arm execution.
10. Output only the Python dictionary.

Planning principles:

- Decompose the task according to object dependencies and inter-arm coordination.
- Respect geometry and reachability when allocating actions to agents.
- Grasp must happen before lift; lift must happen before transfer or placement.
- Synchronize agents within the same stage when cooperative manipulation is required.
- Use U_k_adj for deviations that can be corrected before the stage-critical event.
- Use U_k_re when the stage outcome has failed and the atomic operation must be repeated.
- The termination condition should depend on observable progress in the current stage.

Output format:

```
stage_plans = {
  "stage 1": {
    "u_k": {
      "agent0": "<nominal atomic action>",
      "agent1": "<nominal atomic action>",
      ...
    },
    "gamma_k": "<geometric guidance for monitoring>",
  },
  "psi_k": "<stage termination condition>",
  "U_k_adj": {
    "<deviation case 1>": {
      "agent0": "<corrective atomic action>",
      "agent1": "<corrective atomic action>",
      ...
    },
    ...
  },
  "U_k_re": {
    "<failure case 1>": {
      "agent0": "<re-operation atomic action>"
    },
    ...
  }
}
```

```
"agent1": "<re-operation atomic action>"
> ",
...
}
},
...
}
```

B. Event-triggered 3D geometric monitor

The 3D monitor is a point-cloud predicate checker rather than a learned recovery policy or a VLM-based judge. During execution, CARE continuously tracks the gripper open/close state, but geometric perception is invoked only at stage-critical transitions (e.g., grasp closure or object release) or shortly before a predicted transition when intra-execution adjustment is enabled. This event-triggered design matches the execution protocol used in the main paper.

At each triggered check, SAM 3 [43] extracts masks for the gripper, manipulated object, and target region, while Depth Anything 3 (DA3) [44] provides depth. We fuse these observations into point clouds $\mathcal{P}_{g,t}$, $\mathcal{P}_{o,t}$, and $\mathcal{P}_{tar,t}$ and compute a compact geometric state

$$q_t = \Phi(\mathcal{P}_{g,t}, \mathcal{P}_{o,t}, \mathcal{P}_{tar,t}). \quad (17)$$

Conditioned on the active stage plan Π_{k_t} , the monitor selects

$$\tilde{u}_t = f_{\text{mon}}(\Pi_{k_t}, q_t), \quad \tilde{u}_t \in \{u_{k_t}\} \cup \mathcal{U}_{k_t}^{\text{adj}} \cup \mathcal{U}_{k_t}^{\text{re}}. \quad (18)$$

The same VLA executor subsequently executes nominal, adjustment, and re-operation instructions.

The monitor uses skill-level geometric predicate templates rather than task-specific learned classifiers. Once a predicate template is defined for an atomic skill, it is reused across tasks; only the stage-relevant geometric cues and numerical tolerances are supplied through γ_k and calibrated from rollout statistics.

a) Intra-execution adjustment.: During grasping and approach, the monitor estimates the approach vector $\mathbf{v}_{\text{app}}$ from the recent TCP trajectory and projects the local geometry onto the orthogonal plane $\mathcal{H}_{\perp}$. For global envelope grasping, let $\tilde{p}_{\text{tcp}}$ and $\tilde{\mathcal{L}}_{\text{axis}}$ denote the projected TCP position and object principal axis. Adjustment is triggered when

$$\phi_{\text{geg}} = \mathbb{I} \left[d(\tilde{p}_{\text{tcp}}, \tilde{\mathcal{L}}_{\text{axis}}) > \delta_{\text{geg}} \right]. \quad (19)$$

For local edge grasping, let $\tilde{S}$ be the projected line segment connecting the two fingertips and $\partial\tilde{\mathcal{P}}_o$ the projected object boundary. We trigger adjustment if the fingertips fail to straddle the edge or violate the safety clearance:

$$\phi_{\text{leg}} = \mathbb{I} \left[(\tilde{S} \cap \partial\tilde{\mathcal{P}}_o = \emptyset) \vee \left(\min_{i \in \{1,2\}} d(\tilde{p}_i, \partial\tilde{\mathcal{P}}_o) \leq \delta_{\text{safe}} \right) \right]. \quad (20)$$

For placement, both the effective grasp center p_{gc} and target position p_{target} are projected onto the support plane:

$$\phi_{\text{place}} = \mathbb{I} \left[\|\Pi_{\mathcal{H}_{\text{support}}}(p_{\text{gc}}) - \Pi_{\mathcal{H}_{\text{support}}}(p_{\text{target}})\|_2 > \delta_{\text{place}} \right]. \quad (21)$$

If any corresponding adjustment predicate is active, CARE selects an instruction from $\mathcal{U}_{k_t}^{\text{adj}}$.



Fig. 5. Visualization of point-cloud-based geometric judgement. ✗ indicates cases requiring post-execution re-operation; ! indicates cases requiring intra-execution adjustment; and ✓ indicates correct cases.

b) Post-stage re-operation.: After a stage-critical event, CARE performs discrete verification of the stage outcome. For grasp verification, let p_{obj} denote the object center, p_{gc} the grasp center, $\mathbf{n}_1$ and $\mathbf{n}_2$ the surface normals at the two contact points, and w_{gripper} the measured gripper width. Re-grasp is triggered when

$$\phi_{\text{re-grasp}} = \mathbb{I} \left[\begin{aligned} &\|p_{\text{gc}} - p_{\text{obj}}\|_2 > \delta_{\text{center}} \vee \\ &\mathbf{n}_1^\top \mathbf{n}_2 \geq 0 \vee \\ &w_{\text{gripper}} \leq \epsilon_{\text{empty}} \end{aligned} \right]. \quad (22)$$

For post-placement verification, re-placement is triggered when

$$\phi_{\text{re-place}} = \mathbb{I} \left[\begin{aligned} &\|\Pi_{\mathcal{H}_{\text{support}}}(p_{\text{obj}}^{\text{final}}) - \Pi_{\mathcal{H}_{\text{support}}}(p_{\text{target}})\|_2 \\ &> \delta_{\text{re-place}} \end{aligned} \right]. \quad (23)$$

When either re-operation predicate is active, CARE selects an instruction from $\mathcal{U}_{k_t}^{\text{re}}$.

The geometric tolerances δ_{geg} , δ_{safe} , δ_{place} , δ_{center} , and $\delta_{\text{re-place}}$ are calibrated from stage-conditioned rollout statistics rather than tuned as task-specific learned classifiers. The empty-grasp threshold ϵ_{empty} is determined by the mechanical closing limit of the parallel-jaw gripper.

X. FSR-BENCH DETAILS

FSR-Bench evaluates recovery independently from nominal task execution. Each episode begins from an intermediate failure state and asks the policy to reach the scenario-specific recovery target within a predefined horizon. The benchmark contains 36 recovery scenarios across five tasks: 21 *Easy* local failures that typically admit one corrective operation and 15 *Hard* structural failures that require multi-step correction or coordinated multi-arm recovery.

Table VIII reports the maximum recovery horizon used for each task and regime. The larger hard-regime budgets reflect

TABLE VIII
MAXIMUM RECOVERY STEPS FOR EACH TASK IN FSR-BENCH.

Task	Easy	Hard
Pot Lift Recovery	100	400
Skillet Placement Recovery	200	250
Block Handover Recovery	400	500
Seal Stamp Recovery	200	500
Dual Shoes Recovery	200	400

TABLE IX
FAILURE STATE TYPES AND CORRESPONDING TASKS IN FSR-BENCH.
DSR: DUAL SHOES RECOVERY; SSR: SEAL STAMP RECOVERY; SPR: SKILLET PLACEMENT RECOVERY; BHR: BLOCK HANDOVER RECOVERY; PLR: POT LIFT RECOVERY.

Failure State Type	Tasks
Pose Misalignment	BHR_easy, DSR_easy
Grasp Failure	SSR_easy, BHR_easy, PLR_easy, SPR_easy, DSR_easy
Placement Offset	SSR_easy, DSR_easy
Tipping	SSR_hard, PLR_hard, SPR_hard, DSR_hard
Unreachable Target	SSR_hard, BHR_hard
Target Occupation	SSR_hard, BHR_hard
Accidental Drop	SPR_hard
Carry-over Error	PLR_hard, DSR_hard

the additional atomic operations often required by structural anomalies.

Table IX summarizes the failure taxonomy. *Pose Misalignment* denotes gripper/object or object/target spatial misalignment; *Grasp Failure* includes missed, unstable, or slipping grasps; *Placement Offset* denotes an incorrect final placement; *Tipping* denotes loss of object stability; *Unreachable Target* places the desired target outside the immediately feasible workspace; *Target Occupation* blocks the intended target region with another object; *Accidental Drop* is unintended release during manipulation; *Mis-stacked Objects* violates the

intended stacking relation; and *Carry-over Error* denotes a failure from an earlier stage that remains unresolved and affects subsequent execution. In *Dual Shoes Recovery_hard*, for example, an incorrectly oriented first shoe can remain in the scene while the second placement also fails, yielding a multi-stage recovery problem.

A. Training and test distributions

FSR-Bench deliberately separates the recovery-training distribution from the test distribution. For recovery training, perturbations are sampled *uniformly* within the predefined range of each recovery scenario, and all backbone variants use the same 50 corrective trajectories per recovery type.

For evaluation, we first execute nominal policies without recovery and estimate a stage-conditioned empirical failure distribution

$$p_{\text{fail}}(d \mid k). \quad (24)$$

At test time, we sample $d \sim p_{\text{fail}}(d \mid k)$, inject the deviation at the corresponding stage of a nominal execution, and roll the system forward under physical dynamics. The state after this rollout becomes the actual recovery initialization. Therefore, test episodes are newly generated failure states grounded in observed policy failures rather than memorized replays or direct copies of training perturbations.

Evaluation episodes additionally randomize object geometry, table texture, lighting, distractor objects, and object placement. For a given recovery scenario, all methods are evaluated on the same pre-generated failure initializations.

B. Evaluation protocol

Each method is evaluated over $N_{\text{eval}} = 100$ trials per task–regime pair, with initializations sampled from the corresponding constituent recovery scenarios. Recovery Success Rate (RSR) is

$$\text{RSR} = \frac{1}{N_{\text{eval}}} \sum_{j=1}^{N_{\text{eval}}} \mathbb{I}[r_j = 1], \quad (25)$$

where $r_j = 1$ if the policy reaches the scenario-specific recovery target within its prescribed horizon, and $r_j = 0$ otherwise. For each scenario, the recovery target and horizon are shared across methods. For each task and regime, RSR is computed over 100 trials sampled from its constituent failure states, using the same pre-generated initializations across methods. Figures 15 and 16 visualize all 36 failure states.

XI. ADDITIONAL EXPERIMENTAL ANALYSIS

A. Analysis of corrective interventions

Figure 6 provides an intervention-level view complementary to the component ablation in the main paper. The component ablation separates the effect of corrective training data from the effect of atomic corrective execution, whereas Fig. 6 focuses only on what happens during evaluation after the full CARE model has been trained. Successful recovery

episodes include both intra-execution adjustments and post-stage re-operations, confirming that the two corrective primitives are both used by the closed-loop system. We therefore do not interpret this figure as replacing the Data/Execution component ablation; rather, it illustrates how the learned corrective behaviors are invoked online.

B. Event-triggered monitoring protocol

The VLM-monitor baseline and our 3D monitor use the same triggered observations in the monitoring ablation of the main paper. A check is invoked at stage-critical events such as gripper closure or object release. When a stage supports intra-execution adjustment, CARE additionally checks shortly before a predicted critical transition; the prediction window follows the VLA action horizon of 50. For stages whose boundaries are not directly associated with a gripper-state change, the system performs a check once per 50 executed actions. Using the same trigger schedule isolates the quality and latency of the monitoring mechanism itself rather than differences in invocation frequency.

C. Ablation on the Number of Experience-Guided Rollouts

We ablate the number of preliminary rollouts used to estimate the experience-guided failure distribution. We compare 50, 100, and 150 rollouts and keep the amount of corrective supervision fixed at 50 synthesized corrective atomic trajectories per error type. Table X reports three simulation tasks whose 100-rollout setting is identical to the main-paper training protocol.

Across these tasks, 50 rollouts are consistently weaker, whereas the difference between 100 and 150 rollouts is small. The average success rate is 45.0% with 50 rollouts, 51.5% with 100 rollouts, and 51.0% with 150 rollouts. This supports the use of 100 preliminary rollouts as a practical trade-off between collection cost and the stability of the empirical failure prior.

To analyze the prior directly, we examine *Put Object Cabinet* using the four modeled variables dx , dy , dz , and dr . We compare the distributions estimated from prefixes of 50, 100, 150, and 200 rollouts and track cumulative mean/variance, empirical PDFs, and Jensen–Shannon (JS) divergence to the full-data distribution. At 50 rollouts, the JS divergences are 0.0982, 0.1422, 0.0582, and 0.1556 for dx , dy , dz , and dr , respectively. At 100 rollouts, they drop to 0.0224, 0.0200, 0.0374, and 0.0245; the cumulative statistics also become substantially flatter. The 100-rollout setting provides 92 valid samples for each translational variable and 63 samples for dr . Increasing the budget to 150 rollouts only further reduces the JS divergences to 0.0145, 0.0113, 0.0141, and 0.0174. These results indicate that the failure prior has largely stabilized by 100 rollouts.

D. Ablation on Corrective Recovery Strategies

We further compare CARE with a rollback-based recovery strategy on simulation tasks. Rollback uses the same 3D monitor to detect deviations, restores execution to the final state of the previous atomic stage, and then resumes nominal

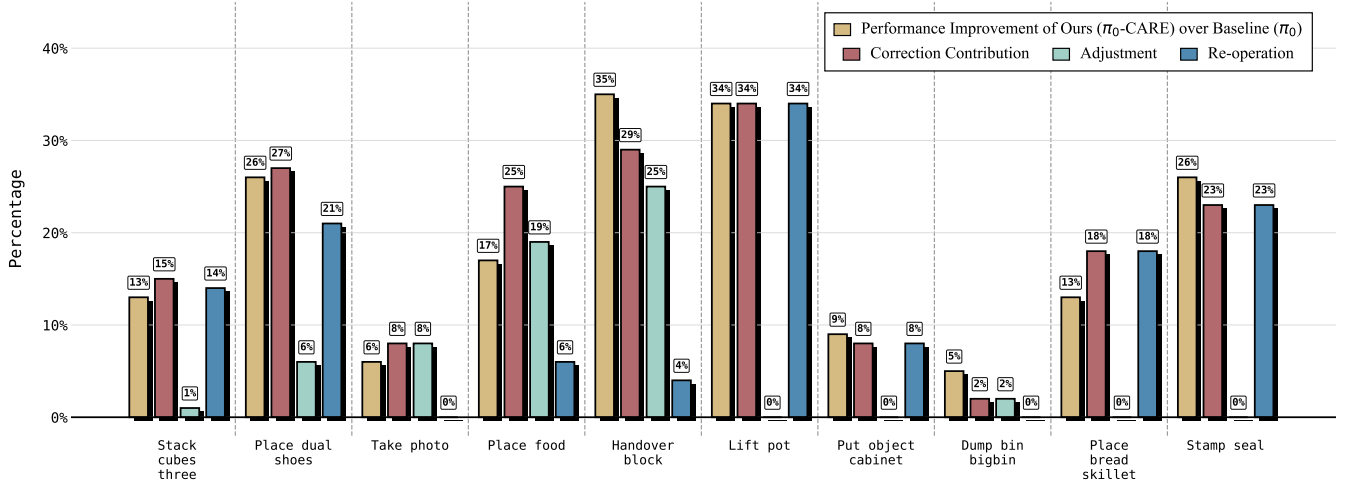


Fig. 6. **Breakdown of successful corrective interventions for π_0 .** *Adjustment* denotes successful intra-execution corrections, while *Re-operation* denotes successful repeated atomic operations after a failed stage outcome.

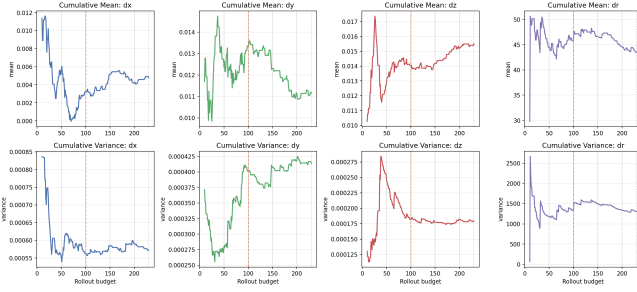


Fig. 7. Cumulative mean and variance of the four modeled variables, dx , dy , dz , and dr , under different rollout budgets on *Put Object Cabinet*. The statistics become much more stable around 100 rollouts, while further increasing the rollout budget yields only limited changes.

execution without issuing an explicit corrective instruction for the current failure. All compared variants use the standard simulation data budget: 150 nominal expert demonstrations and 50 additional trajectories per error type. CARE uses corrective trajectories, while the baseline and rollback variants use matched nominal atomic-stage trajectories.

As shown in Table XI, rollback provides only limited improvement. Returning to an earlier state can reduce accumulated error but does not teach the policy how to act in the current failure state. CARE instead combines explicit corrective supervision with atomic corrective execution and achieves substantially larger gains.

E. Accuracy of the 3D Monitor in Instruction Judgement

We additionally evaluate the 3D monitor on triggered observations from a broader set of simulation and real-world tasks. A monitoring decision is counted as incorrect for either a false positive (a correct execution is judged to require correction) or a false negative (a required correction is missed). The three simulation and two real-world tasks that overlap with the main-paper monitoring ablation use exactly the same reported accuracies. The additional tasks

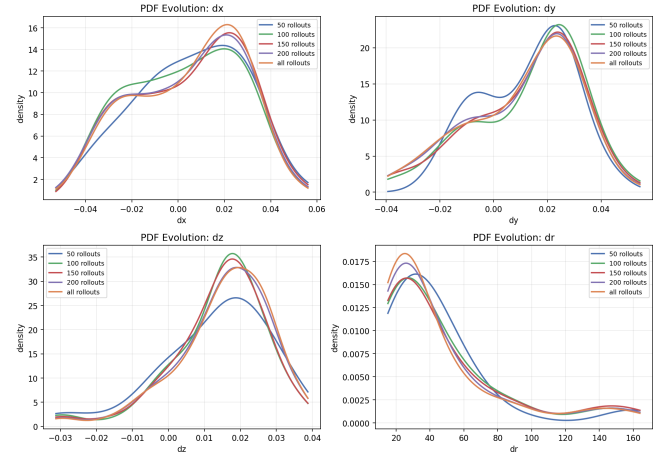


Fig. 8. Evolution of the empirical PDFs of dx , dy , dz , and dr under different rollout budgets on *Put Object Cabinet*. The distributions estimated from 100 rollouts already closely match those obtained with larger rollout budgets.

extend the analysis to object diversity and real-world dual-arm manipulation.

The monitor remains accurate despite substantial object variation. In *Put Object Cabinet*, which contains many object categories, the same skill-level predicate templates achieve 96% monitoring accuracy without a task-specific learned classifier.

XII. TRAINING AND EVALUATION PROTOCOLS

A. Data budgets and VLA training parameters

For standard simulation tasks in RoboTwin 2.0 [6] and RoboFactory [5], each method uses 150 nominal expert demonstrations and 50 additional trajectories per error type. CARE uses experience-guided corrective trajectories for the additional data, whereas the matched baseline uses nominal atomic-stage trajectories of the same scale. The experience-

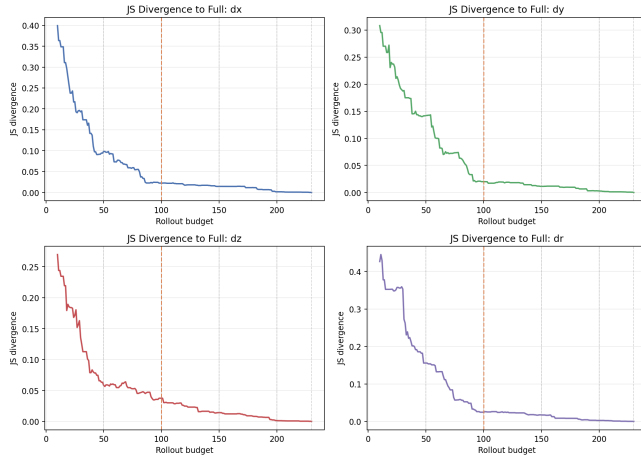


Fig. 9. Jensen-Shannon divergence between the prefix distribution and the full-data distribution for dx , dy , dz , and dr under different rollout budgets on *Put Object Cabinet*. The divergence drops substantially by 100 rollouts and decreases only marginally afterward, indicating that the deviation prior has largely saturated.

TABLE X
ABLATION ON THE NUMBER OF EXPERIENCE ROLLOUTS USED FOR CORRECTIVE DATA SYNTHESIS. THE AMOUNT OF CORRECTIVE SUPERVISION IS FIXED ACROSS ALL SETTINGS.

Task	Backbone	50	100	150
Lift Pot	π_0	83%	91%	87%
	π_0 -FAST	31%	40%	36%
Put Object Cabinet	π_0	68%	68%	72%
	π_0 -FAST	29%	32%	28%
Handover Block	π_0	44%	62%	63%
	π_0 -FAST	15%	16%	20%
Avg.	Overall	45.0%	51.5%	51.0%

guided distributions are estimated from 100 preliminary rollouts.

For the two VLA backbones used by CARE, π_0 and π_0 -FAST, both the vanilla and CARE variants are fine-tuned with the same optimization settings: 40,000 gradient steps, batch size 32, and action horizon 50 on two NVIDIA A800 SXM4 80GB GPUs. The backbone-specific learning-rate settings are given in Tables XV and XVI. ACT, DP, DP3, and RDT-1B are trained using their model-specific benchmark recipes; their configurations are listed for reproducibility in Tables XVII–XX.

For FSR-Bench, the data and optimization protocol is intentionally unified across backbones: each recovery type uses 50 uniformly sampled corrective trajectories, and all variants are trained for 20,000 steps with batch size 32 and action horizon 50. CARE and its corresponding vanilla backbone use identical hyperparameters so that the comparison isolates atomic corrective execution.

TABLE XI
COMPARISON OF ROLLBACK-BASED RECOVERY AND CARE ON THREE SIMULATION TASKS USING THE π_0 BACKBONE.

Task	π_0	π_0 (Rollback)	π_0 (CARE)
Lift Pot	57%	67%	91%
Place Dual Shoes	14%	18%	40%
Stack Cubes Three	48%	52%	61%
Average	39.7%	45.7%	64.0%

TABLE XII
ACCURACY OF THE SKILL-LEVEL 3D MONITOR ON SIMULATION AND REAL-WORLD TASKS. RESULTS FOR LP, PDS, HB, PTT, AND STB ARE IDENTICAL TO THOSE IN THE MAIN-PAPER MONITORING ABLATION.

Task	Acc.	Task	Acc.
Lift Pot	98%	Place Bread Skillet	92%
Place Dual Shoes	95%	Handover Block	96%
Dump Bin Bigbin	94%	Put Object Cabinet	96%
Stamp Seal	96%	Stack Two Cubes	90%
Pass Two Toys	86%	Stack Two Bowls	94%
Place Two Cubes	88%		
Average over all listed tasks		93.2%	

B. Real-world setup and evaluation

The real-world platform is a dual-arm LeRobot SO-101 system with 12 DoF in total (6 per arm), one global RGB camera, and one wrist RGB camera per arm; all cameras operate at 30Hz. For each real-world task, we collect 50 nominal demonstrations and 20 additional trajectories per error type. CARE uses corrective demonstrations for the additional data, while the matched baseline follows the same data-budget protocol. We use 100 preliminary executions per task to estimate the empirical failure distribution and evaluate standard task success over 100 trials per task.

The VLA policy is served from an A800 server during robot execution. SAM 3 [43] and DA3 [44] run locally on an RTX 4090 to provide masks and depth for geometric monitoring. Separating policy inference from perception keeps the event-triggered monitor responsive during deployment.

C. Evaluation coverage and action vocabulary

We evaluate seven hard bimanual tasks from RoboTwin 2.0, three multi-arm collaboration tasks from RoboFactory, five FSR-Bench recovery tasks, and four real-world dual-arm tasks. RoboTwin 2.0 uses the `demo_randomized` setting for both training and evaluation. FSR-Bench uses 100 trials per task–regime pair, while the standard real-world tasks use 100 evaluation trials per task.

Across tasks, the VLM planner uses a shared atomic-action vocabulary, with object-specific arguments inserted into the language instruction. Table XIV lists representative action types and instruction forms.

TABLE XIII

UNIFIED FSR-BENCH TRAINING PROTOCOL USED BY RDT-1B,
 π_0 -FAST, AND π_0 VARIANTS. BACKBONE-SPECIFIC OPTIMIZER
 SETTINGS FOLLOW THE CORRESPONDING MODEL IMPLEMENTATION.

Hyperparameter	Value
Corrective trajectories / recovery type	50 (uniformly sampled)
Training steps	20,000
Batch size	32
Action horizon	50
Baseline/CARE optimization	Identical within each backbone

TABLE XIV

UNIFIED ACTION VOCABULARY AFTER MERGING OBJECT-SPECIFIC
 VARIATIONS.

Action Type	Example Instruction
Wait	Wait for the other robots.
Grasp	Grasp the camera / bread / drawer bar / object.
Lift	Lift the object / camera / meat.
Place	Place the object / on the target / into the cabinet.
Re-grasp	Re-grasp the object.
Re-place	Re-place the object.
Adjust	Adjust the grasping / placement / press position.
Handover	Handover the object.
Press	Press the camera shutter button.
Move	Move the pot to the middle.
Pull	Pull the drawer.
Dump	Dump the dustbin.
Rotate	Rotate it back to the upright orientation.
Remove obstacle	Remove the obstacle.
Step back and wait	Step back and wait for the other robots. / Open the gripper and retreat.

XIII. VISUALIZATION

We visualize the nominal execution and corrective execution cases for *Place Dual Shoes*. The nominal trajectory is shown in Fig. 11, while the corrective processes for Stages 1–3 are shown in Figs. 12–14. Additional qualitative results for the remaining tasks are provided in the supplementary demo video.

TABLE XV

TRAINING CONFIGURATION OF π_0 -FAST IN ROBOTWIN AND
 ROBOFACTORY.

Hyperparameter	Value
Model	π_0 -FAST
Initialization	Base checkpoint initialization
Action horizon	50
Maximum token length	280
Batch size	32
Training steps	40,000
Peak learning rate	5×10^{-5}
Warmup steps	200
Decay steps	1400
Hardware	$2 \times$ NVIDIA A800 GPUs

TABLE XVI

TRAINING CONFIGURATION OF π_0 IN ROBOTWIN AND ROBOFACTORY.

Hyperparameter	Value
Model	π_0
Initialization	Base checkpoint initialization
Action horizon	50
Maximum token length	180
Batch size	32
Training steps	40,000
Peak learning rate	3×10^{-5}
Warmup steps	300
Decay steps	10,000
Final learning rate	1×10^{-6}
Hardware	$2 \times$ NVIDIA A800 GPUs

TABLE XVII

TRAINING CONFIGURATION OF DP IN ROBOTWIN AND ROBOFACTORY.

Hyperparameter	Value
Model	DP
Policy architecture	Diffusion U-Net image policy
Observation encoder	Multi-image ResNet-18 encoder
Horizon	8
Observation steps	3
Action steps	6
Diffusion steps (train / infer)	100 / 100
Batch size	512
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-6}
LR scheduler	Cosine
Warmup steps	500
Training epochs	600
Hardware	$1 \times$ GPU (cuda:0)

TABLE XVIII
TRAINING CONFIGURATION OF DP3 IN ROBOTWIN.

Hyperparameter	Value
Model	DP3
Observation encoder	PointNet encoder
Horizon	8
Observation steps	3
Action steps	6
Crop shape	80×80
Diffusion steps (train / infer)	100 / 10
Noise scheduler	DDIM
Batch size	3096
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-6}
LR scheduler	Cosine
Warmup steps	500
Training epochs	3000
Hardware	$1 \times \text{GPU (cuda:0)}$

TABLE XIX
TRAINING CONFIGURATION OF ACT IN ROBOTWIN.

Hyperparameter	Value
Model	ACT
KL weight	10
Chunk size	50
Hidden dimension	512
Feedforward dimension	3200
Batch size	64
Learning rate	1×10^{-5}
Training epochs	6000

TABLE XX
TRAINING CONFIGURATION OF RDT-1B IN ROBOTWIN.

Hyperparameter	Value
Model	RDT-1B
Train batch size	32
Sample batch size	64
Max training steps	20000
Learning rate	1×10^{-4}
Dataloader workers	8
State noise SNR	40
Gradient accumulation steps	1
Hardware	$1 \times \text{GPU (cuda:0)}$

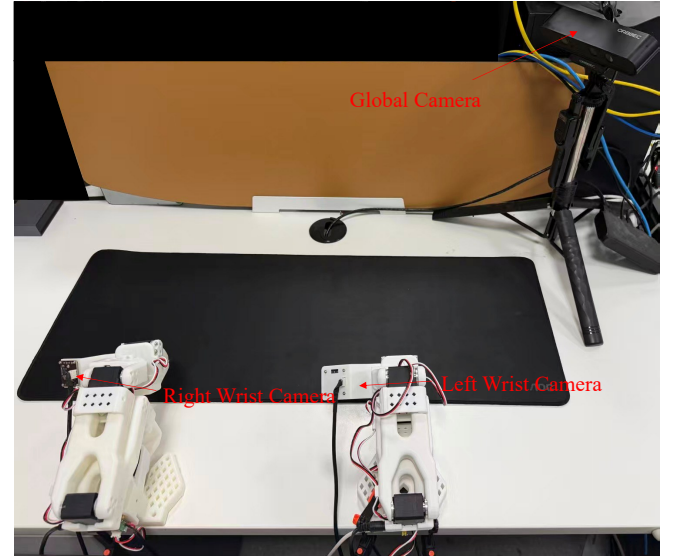


Fig. 10. Real-world setup based on the dual-arm LeRobot SO-101 platform.

- Stage1——agent0: Grasp the left shoe.
agent1: Grasp the right shoe.



- Stage2——agent0: Place the left shoe.
agent1: Wait for the other robot.

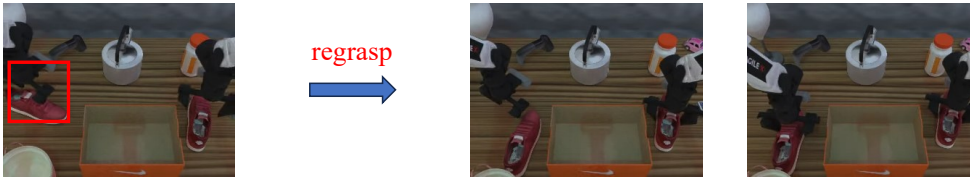


- Stage3——agent0: Wait for the other robot.
agent1: Place the right shoe.

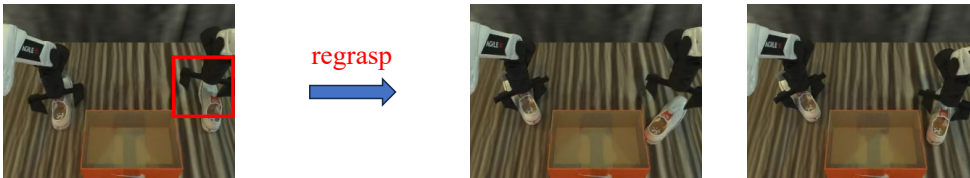


Fig. 11. Visualization of the nominal execution for *Place Dual Shoes* (Part 1/4). We show the full process of the three nominal atomic actions in the task.

- Stage1——agent0: Grasp the left shoe.
agent1: Grasp the right shoe.
- agent0: Lift slightly and regrasp the left shoe.
agent1: Wait for the other robot.



- Stage1——agent0: Grasp the left shoe.
agent1: Grasp the right shoe.
- agent0: Wait for the other robot.
agent1: Lift slightly and regrasp the right shoe.



- Stage1——agent0: Grasp the left shoe.
agent1: Grasp the right shoe.
- agent0: Lift slightly and regrasp the left shoe.
agent1: Lift slightly and regrasp the right shoe.

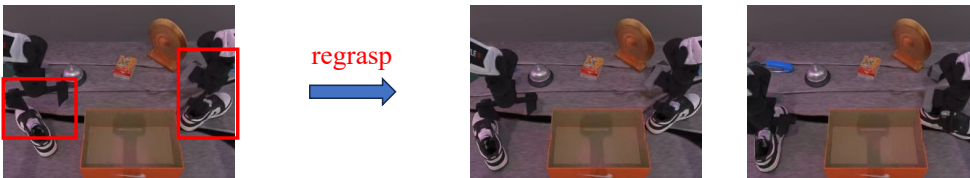
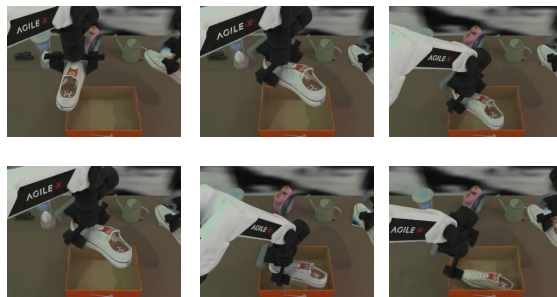


Fig. 12. Visualization of the corrective execution for *Place Dual Shoes* at Stage 1 (Part 2/4). The figure shows the correction process for the first stage of the task.

- Stage2——agent0: Place the left shoe.
agent1: Wait for the other robot.



adjust

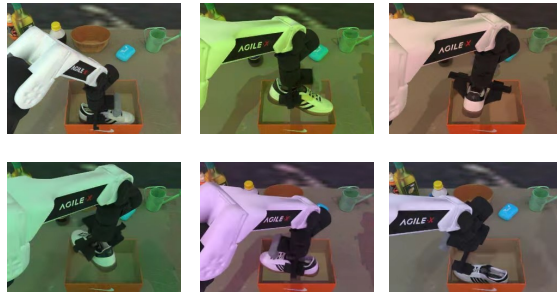


- agent0: Adjust the placement position.
agent1: Wait for the other robot.

- Stage2——agent0: Place the left shoe.
agent1: Wait for the other robot.



replace



- agent0: Lift slightly and replace the left shoe.
agent1: Wait for the other robot.

Fig. 13. Visualization of the corrective execution for *Place Dual Shoes* at Stage 2 (Part 3/4). We show the correction process corresponding to the second atomic stage of the task.

- Stage3——agent0: Wait for the other robot.
agent1: Place the right shoe.



adjust



- agent0: Wait for the other robot.
agent1: Adjust the placement position.

- Stage3——agent0: Wait for the other robot.
agent1: Place the right shoe.



replace



- agent0: Wait for the other robot.
agent1: Lift slightly and replace the right shoe.

Fig. 14. Visualization of the corrective execution for *Place Dual Shoes* at Stage 3 (Part 4/4). We show the correction process corresponding to the third atomic stage of the task.

Pot Lift Recovery_easy



Grasp Failure of left arm



Grasp Failure of right arm



Grasp Failure of both arms

Pot Lift Recovery_hard



Carry-over Error



Tipping

Block Handover Recovery_easy



Pose Misalignment of left arm



Pose Misalignment of right arm



Grasp Failure of left arm



Grasp Failure of right arm

Block Handover Recovery_hard



Target Occupation



Unreachable Target

Skillet Placement Recovery_easy



Grasp Failure of left arm



Grasp Failure of right arm



Grasp Failure of both arms

Skillet Placement Recovery_hard



Accidental Drop



Tipping

Dual Shoes Recovery_easy



Grasp Failure of left arm



Grasp Failure of right arm



Grasp Failure of both arms



Pose Misalignment of left arm



Pose Misalignment of right arm



Placement Offset of left arm



Placement Offset of right arm

Fig. 15. Visualization of the 36 failure states in FSR-Bench (Part 1/2). The benchmark covers 21 easy and 15 hard cases spanning local deviations and structural failures.

Dual Shoes Recovery _hard



Tipping of left arm



Tipping of right arm



Carry-over Error

Seal Stamp Recovery _easy



Grasp Failure of left arm



Grasp Failure of right arm



Placement Offset of left arm



Placement Offset of right arm

Seal Stamp Recovery _hard



Tipping of left arm



Tipping of right arm



Unreachable Target of left arm



Unreachable Target of right arm



Target Occupation of left arm



Target Occupation of right arm

Fig. 16. Visualization of the 36 failure states in FSR-Bench (Part 2/2).